Sentinelleai — Audit Report

MagnetaETF.sol · 9a57a84f-cc4a-42af-b365-d50e359dd768 · 2026-05-22 20 h 38 min 46 s

CAUTION 52 / 100 5 consolidated findings

MagnetaETF presents two confirmed HIGH-severity vulnerabilities and two MEDIUM-severity issues. The most critical is the pervasive use of balanceOf(address(this)) across all pricing paths (mint, redeem, previewMint, previewRedeem), confirmed by the static sentinel, developer, historian, and formalist agents — this is a textbook SC02 (Price Manipulation / Oracle Manipulation) vector matching the Venus Protocol 2026 exploit pattern, where direct ERC-20 transfers can skew exchange rates without going through the deposit mechanism. A secondary MEDIUM concern is read-only reentrancy on previewRedeem (SC08), confirmed by sentinel, historian, and formalist, which could be exploited by external integrators or oracles during callbacks. The mint() function's open access (SC01) is partially by design for Community bootstrap mode but warrants explicit documentation and consideration of mode-gating. The static sentinel's keccak256/RNG findings (SC09) are false positives: the hashes are deterministic action identifiers for a timelock system, not random number generation — this is explicitly refuted by the developer agent and confirmed by code review. The bootstrap() function's conditional access control is intentional per the documented dual-mode design. No reentrancy in state-mutating functions was found due to correct CEI ordering and nonReentrant guards. The contract requires remediation of the internal accounting vulnerability before production deployment.

Severity breakdown

1 HIGH 2 MEDIUM 1 LOW 1 INFO

Static sentinel pre-scan

Static sentinels matched 11 pattern(s) — 9 HIGH, 2 MEDIUM.

MEDIUM SC01 · AC-2 · line 173

Privileged function lacks any modifier

Functions named *withdraw / *upgrade / *setOwner / *mint / *pause without onlyOwner / onlyRole / authority modifier are likely missing access control.

```
function mint(uint256 etfAmount) external nonReentrant whenNotPaused {
```

MEDIUM SC08 · REE-2 · line 243

View function used externally not protected against read-only reentrancy

Public view functions (get_virtual_price / getRate / previewWithdraw) consumed by external oracles can return stale state during a callback. Add nonReentrantView modifier or pre-call the protected function with zero args.

```
function previewRedeem(uint256 etfAmount) external view returns (uint256[] memory amounts) {
```

HIGH SC02 · ACC-1 · line 182

balanceOf(this) used in pricing / rate calculation

Using token.balanceOf(address(this)) as an input to swap/redeem/getRate is bypassable: an attacker can ERC20-transfer tokens directly to the contract (skipping deposit) and skew the calculation. Track an internal accounting variable instead.

```
uint256 vaultBalance = IERC20(_assets[i].token).balanceOf(address(this));
```

HIGH SC02 · ACC-1 · line 202

balanceOf(this) used in pricing / rate calculation

Using token.balanceOf(address(this)) as an input to swap/redeem/getRate is bypassable: an attacker can ERC20-transfer tokens directly to the contract (skipping deposit) and skew the calculation. Track an internal accounting variable instead.

```
uint256 vaultBalance = IERC20(_assets[i].token).balanceOf(address(this));
```

HIGH SC02 · ACC-1 · line 229

balanceOf(this) used in pricing / rate calculation

Using token.balanceOf(address(this)) as an input to swap/redeem/getRate is bypassable: an attacker can ERC20-transfer tokens directly to the contract (skipping deposit) and skew the calculation. Track an internal accounting variable instead.

```
uint256 vaultBalance = IERC20(_assets[i].token).balanceOf(address(this));
```

HIGH SC02 · ACC-1 · line 249

balanceOf(this) used in pricing / rate calculation

Using token.balanceOf(address(this)) as an input to swap/redeem/getRate is bypassable: an attacker can ERC20-transfer tokens directly to the contract (skipping deposit) and skew the calculation. Track an internal accounting variable instead.

```
uint256 vaultBalance = IERC20(_assets[i].token).balanceOf(address(this));
```

HIGH SC09 · RNG-2 · line 273

keccak256 mixes block-derived entropy (predictable seed)

Any combination of block.number / block.timestamp / block.blockhash / block.coinbase / block.difficulty / blockhash() / now inside a keccak256 is predictable by the proposer (or trivially by anyone within the same block) and is not a safe RNG source. Use Chainlink VRF, RANDAO with delay, or commit-reveal.

```
bytes32 actionId = keccak256(abi.encode("updateWeights", newWeights));
```

HIGH SC09 · RNG-2 · line 287

keccak256 mixes block-derived entropy (predictable seed)

Any combination of block.number / block.timestamp / block.blockhash / block.coinbase / block.difficulty / blockhash() / now inside a keccak256 is predictable by the proposer (or trivially by anyone within the same block) and is not a safe RNG source. Use Chainlink VRF, RANDAO with delay, or commit-reveal.

```
bytes32 actionId = keccak256(abi.encode("updateWeights", newWeights));
```

HIGH SC09 · RNG-2 · line 325

keccak256 mixes block-derived entropy (predictable seed)

Any combination of block.number / block.timestamp / block.blockhash / block.coinbase / block.difficulty / blockhash() / now inside a keccak256 is predictable by the proposer (or trivially by anyone within the same block) and is not a safe RNG source. Use Chainlink VRF, RANDAO with delay, or commit-reveal.

```
bytes32 actionId = keccak256(abi.encode("closeETF"));
```

HIGH SC09 · RNG-2 · line 339

keccak256 mixes block-derived entropy (predictable seed)

Any combination of block.number / block.timestamp / block.blockhash / block.coinbase / block.difficulty / blockhash() / now inside a keccak256 is predictable by the proposer (or trivially by anyone within the same block) and is not a safe RNG source. Use Chainlink VRF, RANDAO with delay, or commit-reveal.

```
bytes32 actionId = keccak256(abi.encode("closeETF"));
```

HIGH SC01 · AST-MISSING-ACCESS-CONTROL · line 146

External state-mutating function lacks access control

An external/public, non-view function that writes to state without an onlyOwner / role / msg.sender check is callable by anyone. Even if the name doesn't match a privileged-keyword whitelist, the lack of any gate is suspicious.

```
function bootstrap(uint256[] calldata amounts) external nonReentrant whenNotPaused {
require(!isBootstrapped, "MagnetaETF: already bootstrapped"); if (bootstrapMode ==
BootstrapMode.Creator) { require
```

Consolidated findings

HIGH CVSS 8.1 SC02: Price Oracle / Rate Manipulation

balanceOf(address(this)) used in all pricing calculations — exchange rate manipulation

The contract uses IERC20(_assets[i].token).balanceOf(address(this)) as the sole source of truth for vault balances in mint(), redeem(), previewMint(), and previewRedeem(). An attacker can directly transfer tokens to the contract address (bypassing the deposit path) to inflate the apparent vault balance, causing subsequent minters to overpay or redeemers to receive inflated amounts. This matches the Venus Protocol March 2026 root cause ($2M+) where direct ERC-20 transfers bypassed deposit accounting and manipulated exchange rates. The attack is especially impactful in the mint() path where rounding-up logic amplifies the effect.

```
Lines 182, 202, 229, 249 — mint(), previewMint(), redeem(), previewRedeem()
```

→ Introduce an internal mapping (e.g., mapping(address ⇒ uint256) private _vaultBalances) updated exclusively within bootstrap(), mint(), and redeem(). Replace all balanceOf(address(this)) calls in rate calculations with this internal accounting variable. balanceOf may still be used for sanity checks but must not drive pricing logic.

Confirmed by: static-sentinel, developer, historian, formalist

MEDIUM CVSS 5.3 SC08: Read-Only Reentrancy

Read-only reentrancy on previewRedeem() and previewMint() view functions

previewRedeem() and previewMint() are external view functions that read balanceOf(address(this)) to compute rates. If an external protocol (e.g., a lending market or aggregator) calls these functions during a callback triggered by a token transfer within a state-mutating transaction, the returned values may reflect an intermediate, inconsistent state. This matches the dForce 2023 ($3.7M) and Market.xyz 2023 ($220K) read-only reentrancy patterns. The risk is amplified if any underlying asset is a rebasing or hook-enabled token.

```
Lines 199–207 (previewMint), Lines 243–253 (previewRedeem)
```

→ Add a nonReentrantView modifier (using a transient storage slot or the ReentrancyGuard's _status check in view mode) to both preview functions. Alternatively, document prominently that these functions must not be used as on-chain price oracles by external protocols. Resolving the internal accounting issue (finding #1) would also mitigate this by removing the balanceOf dependency.

Confirmed by: static-sentinel, historian, formalist, developer

MEDIUM CVSS 4.8 SC01: Access Control

mint() function lacks access control — open minting after bootstrap

The mint() function has no onlyOwner or role-based modifier, allowing any address to mint ETF tokens after bootstrap by depositing proportional assets. While this is consistent with an open ETF design, it is inconsistent with the Creator bootstrap mode intent (where only the owner funds the initial vault). In Creator mode, there is no mechanism to restrict subsequent minting to authorized parties, which may not match operator expectations. The static sentinel flagged this as a missing access control pattern (AC-2). The developer agent confirms open minting is intentional but notes it should be documented.

```
Line 173 — function mint()
```

→ If open minting is intended for all modes, add explicit NatSpec documentation stating this. If Creator mode should restrict minting to the owner or a whitelist, add a conditional check: if (bootstrapMode == BootstrapMode.Creator) { require(msg.sender == owner(), ...); }. Consider adding a mint cap or rate limiter regardless of mode.

Confirmed by: static-sentinel, historian, formalist, developer

LOW CVSS 2.1 SC01: Access Control

bootstrap() has conditional access control — intentional but warrants documentation

The bootstrap() function allows anyone to call it in Community mode, while restricting it to the owner in Creator mode. The static sentinel flagged this as a missing access control issue (AST-MISSING-ACCESS-CONTROL). However, this is the documented and intended dual-mode design. The one-time isBootstrapped guard prevents repeated calls. The risk is low but the conditional logic could confuse auditors or integrators.

```
Line 146 — function bootstrap()
```

→ Add explicit NatSpec comments clarifying that Community mode intentionally allows any first depositor. No code change required. Consider emitting the bootstrapMode in the Bootstrapped event for on-chain transparency.

Confirmed by: static-sentinel, formalist

INFO CVSS 0.0 SC09: Randomness (False Positive)

keccak256 action ID generation flagged as RNG — confirmed false positive

The static sentinel flagged keccak256(abi.encode('updateWeights', newWeights)) and keccak256(abi.encode('closeETF')) at lines 273, 287, 325, and 339 as predictable RNG. These are deterministic action identifiers for a timelock system, not random number generation. The hashes are intentionally deterministic and predictable — that is a feature, not a bug, as it allows the same action to be consistently identified across queue and execute calls. No exploit vector exists from this pattern.

Lines 273, 287, 325, 339

→ No action required. The static sentinel rule (RNG-2/SC09) does not apply to deterministic action ID generation. Consider adding a comment in the code noting that these hashes are intentionally deterministic identifiers.

Confirmed by: developer

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL `access-control` ethereum 2025-05-28 **$12 000 000**

Corkprotocol — Access Control (2025-05-28)

https://x.com/SlowMist_Team/status/1928100756156194955

HIGH `access-control` bsc 2023-03-28 **$8 900 000**

SafeMoon Hack — Access Control (2023-03-28)

https://twitter.com/zokyo_io/status/1641014520041840640

HIGH `access-control` linea 2024-06-01 **$6 880 000**

VeloCore — lack-of-access-control (2024-06-01)

https://x.com/BeosinAlert/status/1797247874528645333

CRITICAL `reentrancy` ethereum 2022-04-30 **$80 000 000**

Rari Capital/Fei Protocol — Flashloan Attack + Reentrancy (2022-04-30)

https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc

CRITICAL `reentrancy` fantom 2021-12-18 **$30 000 000**

Grim Finance — Flashloan & Reentrancy (2021-12-18)

https://cointelegraph.com/news/defi-protocol-grim-finance-lost-30m-in-5x-reentrancy-hack

CRITICAL `reentrancy` ethereum 2020-04-19 **$25 000 000**

LendfMe — ERC777 Reentrancy (2020-04-19)

https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09

CRITICAL `oracle` ethereum 2021-10-27 **$130 000 000**

CreamFinance — Price Manipulation (2021-10-27)

https://medium.com/immunefi/hack-analysis-cream-finance-oct-2021-fc222d913fc5

CRITICAL `oracle` arbitrum 2025-07-09 **$41 000 000**

GMX — Share price manipulation (2025-07-09)

https://x.com/GMX_IO/status/1943336664102756471

CRITICAL `oracle` polygon 2021-11-30 **$31 000 000**

MonoX Finance — Price Manipulation (2021-11-30)

https://slowmist.medium.com/detailed-analysis-of-the-31-million-monox-protocol-hack-574d8c44a9c8

HIGH `donation` zksync 2025-02-27

Venus_ZKSync — Donation Attack (2025-02-27)

https://explorer.zksync.io/

tx/0x35a0172fb6bd450ceb29aa67dc85221826dfd0b7528375400b4ccf15c1eed0d8

> **HIGH** `logic-error` fantom 2022-03-09 **$2 600 000**
>
> Fantasm Finance — Business logic in mint() (2022-03-09)
>
> https://twitter.com/fantasm_finance/status/1501569232881995785

> **MEDIUM** `logic-error` avalanche 2022-12-13 **$845 000**
>
> - ElasticSwap — Business Logic Flaw (2022-12-13)
>
> https://quillaudits.medium.com/decoding-elastic-swaps-850k-exploit-quillaudits-9ceb7fcd8d1a

---

Generated by Sentinelleai on 2026-05-23T00:42:10.795Z. Audit ID: `9a57a84f-cc4a-42af-b365-d50e359dd768`.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.